# AP3-4 · MVC con routing y Composer

## Objetivos

- Sustituir el autoload manual por **Composer** (PSR-4).
- Pasar del routing por query string a rutas por **path** (más cercanas a Symfony).
- Ampliar la aplicación: crear y eliminar tareas.
- Aplicar principios SOLID (interfaces en el Core).

## Recursos

- Tema 3 (namespaces, Composer, Singleton).
- Actividad previa: AP3-3.
- Script SQL: `todolist.sql`.

## Enunciado

Partiendo de la idea de AP3-3, construye la misma aplicación de tareas con estos cambios:

1. **Elimina** el autoload casero y usa Composer (PSR-4). En el `index.php` deberás llamar a:

```php
require_once "../vendor/autoload.php";
```

2. El routing deja de basarse en `?ruta=` y pasa a interpretar el **path** de la URL.

3. Para poder utilizar este nuevo routing, modifica el archivo `Request` por:

```
// Lo primero que debemos hacer es obtener la ruta del navegador mediante
        la URI
$rawRoute = $_SERVER["REQUEST_URI"];
// Separamos la URI en las diferentes partes que contiene, separadas por la
        "/" de las carpetas
// y lo guardamos en un array.
$rawRouteElements = explode("/", $rawRoute);
// El primer elemento (0) lo descartamos porque es el texto de la URI que
        se encuentra antes de la "/"
// Nos quedamos con el segundo elemento (1) que definimos como la ruta que
        va a usar nuestra aplicación
$this->route = "/" . $rawRouteElements[1];
// Guardamos todos los parámetros que hayamos recibido dentro de un nuevo
        array que comienza por el elemento tercero (2)
$this->params = array_slice($rawRouteElements, 2);
```

4. Crea las siguientes rutas en `routes.json` y añade las funcionalidades.

| Ruta | Funcionalidad |
|---|---|
| `/` | Listado (título y fecha de vencimiento) |
| `/detalle` | Detalle completo de un registro. El parámetro es el **slug** (no el `id`) |
| `/addTask` | Formulario de alta |
| `/saveTask` | Procesa el POST, guarda la tarea y vuelve al listado |
| `/deleteTask` | Elimina una tarea por **id** numérico y vuelve al listado |

   Ejemplos de URL:

| URL | Comportamiento |
|---|---|
| `http://localhost:8001/` | Listado |
| `http://localhost:8001/detalle/configurar-entorno-docker` | Detalle por slug |
| `http://localhost:8001/addTask` | Formulario de alta |
| `http://localhost:8001/saveTask` | Guardado (POST del formulario) |
| `http://localhost:8001/deleteTask/3` | Borrado por id |

5. Mantén Singleton + repositorio sobre la tabla `task`.
6. Define las rutas en configuración; el front controller solo arranca el despachador.
7. Se recomienda refactorizar siguiendo principios SOLID (por ejemplo, interfaces en el Core: conexión, request, colección de rutas) y tipar el despachador con ellas cuando puedas.

Campos mínimos del formulario de alta: título y descripción. Al guardar, genera (o guarda) también el `slug` a partir del título.

## Criterios de aceptación

- `composer dump-autoload` (o `install`) y la app arranca cargando `vendor/autoload.php`.

- Funcionan: listado, detalle por path con **slug**, alta (formulario + guardado) y borrado por **id**.
- Tras alta o borrado se vuelve al listado.
- No queda el `autoload.php` manual de AP3-3.
- Las URLs de detalle no exponen el `id` numérico; las de borrado sí usan el `id`.

## Cómo probarlo

Desde la carpeta del proyecto ( `AP3-4` ):

```
composer dump-autoload
```

Desde la raíz del repositorio (Docker del curso):

```
docker compose exec php php -S 0.0.0.0:8000 -t bloque_1/03-MVC/AP3-4/public
```

Abre `http://localhost:8001/` y comprueba listado, detalle, alta y borrado.

## Preguntas de cierre

1. ¿Qué problema concreto resuelve Composer respecto al autoload de AP3-3?
2. ¿Por qué `/detalle/configurar-entorno-docker` es más "SEO" (y legible) que `/?ruta=detalle&params=1` ?
3. ¿Qué ganas tipando el despachador con interfaces en vez de clases concretas?